TASK 1

TITANIC SURVIVAL PREDICTION

Machine Learning Classification Project

Prepared using Python, Pandas, Seaborn and Scikit-learn

Project workflow: Dataset loading • Data cleaning • Visualization • Model training • Evaluation

1. Introduction

This project uses machine learning to predict whether a passenger survived the Titanic disaster. Features include passenger class, sex, age, family counts, fare and embarkation port. The target variable is survived (0 = did not survive, 1 = survived).

2. Objectives

• Load the classic Titanic dataset.

• Handle missing numerical and categorical values.

• Explore survival patterns using charts.

• Train a Logistic Regression classifier.

• Evaluate accuracy, precision and recall.

3. Tools and Libraries

Tool	Purpose
Python	Programming language
Pandas / NumPy	Data handling
Matplotlib / Seaborn	Charts and visual exploration
Scikit-learn	Preprocessing, training and evaluation
Logistic Regression	Binary classification model

4. Dataset and Preprocessing

The code loads the Titanic dataset available through Seaborn. It selects survived as the target and pclass, sex, age, sibsp, parch, fare and embarked as input features. Missing numerical values are filled using the median; missing categorical values use the most frequent category. Sex and embarked are one-hot encoded, and numerical features are standardized. Cabin is not included because it has many missing entries in this version of the dataset.

5. Exploratory Data Analysis

Three charts are generated: survival rate by gender, survival rate by passenger class, and age distribution by survival outcome. In the classic Titanic data, women generally had higher survival rates than men, and first-class passengers generally had higher survival rates than passengers in lower classes. These are associations in the dataset, not proof of causation.

6. Model Training

The dataset is split into 80% training data and 20% test data, using stratification to preserve the survival-class proportions. Logistic Regression is trained in a Scikit-learn pipeline so preprocessing is learned from the training data only, reducing data leakage.

7. Evaluation Metrics

Accuracy is the proportion of all test passengers classified correctly. Precision is the proportion of passengers predicted to survive who actually survived. Recall is the proportion of actual survivors that the model correctly identifies. The Python script prints these metrics, a classification report and a confusion matrix after running.

8. Findings and Conclusion

The expected exploratory pattern is that gender and passenger class are informative predictors of survival. The trained model provides a baseline for binary classification. Exact accuracy, precision and recall depend on the dataset version, split and library versions, so the report intentionally does not invent scores; run the included script to obtain the measured values. Further improvements could compare Random Forest, tune model settings and validate results with cross-validation.

9. How to Run

Install dependencies with: pip install pandas numpy matplotlib seaborn scikit-learn. Then run: python titanic_survival_prediction.py. Internet access may be needed the first time Seaborn downloads the dataset. The script saves titanic_visualizations.png and titanic_test_predictions.csv in the current folder.

Appendix: Complete Python Code

```
# Titanic Survival Prediction
# Install dependencies if needed:
# pip install pandas numpy matplotlib seaborn scikit-learn

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report, confusion_matrix

# 1. Load the classic Titanic dataset from Seaborn
df = sns.load_dataset("titanic")
print("Dataset shape:", df.shape)
print(df.head())

# Keep useful passenger features and survival target
data = df[["survived", "pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]].copy()

# 2. Clean data and encode categorical features
print("\\nMissing values before cleaning:\\n", data.isnull().sum())
X = data.drop(columns="survived")
y = data["survived"]

numeric_features = ["pclass", "age", "sibsp", "parch", "fare"]
categorical_features = ["sex", "embarked"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])

# 3. Visual exploration
sns.set_theme(style="whitegrid")
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.barplot(data=data, x="sex", y="survived", ax=axes[0], errorbar=None)
axes[0].set_title("Survival Rate by Gender")
axes[0].set_ylabel("Survival rate")
sns.barplot(data=data, x="pclass", y="survived", ax=axes[1], errorbar=None)
axes[1].set_title("Survival Rate by Passenger Class")
axes[1].set_ylabel("Survival rate")
sns.histplot(data=data, x="age", hue="survived", bins=25, multiple="stack", ax=axes[2])
axes[2].set_title("Age Distribution by Survival")
plt.tight_layout()
plt.savefig("titanic_visualizations.png", dpi=200, bbox_inches="tight")
plt.show()

# 4. Train a classification model (Logistic Regression)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

```
# 5. Evaluate the model
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions, zero_division=0)
recall = recall_score(y_test, predictions, zero_division=0)
print("\\nEvaluation results")
print(f"Accuracy : {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall   : {recall:.3f}")
print("\\nClassification report:\\n", classification_report(y_test, predictions, zero_division=0))
print("Confusion matrix:\\n", confusion_matrix(y_test, predictions))

# Save the cleaned dataset and test predictions
results = X_test.copy()
results["actual_survival"] = y_test
results["predicted_survival"] = predictions
results.to_csv("titanic_test_predictions.csv", index=False)
print("\\nSaved: titanic_visualizations.png and titanic_test_predictions.csv")
```